

Mục lục

Chương 1: Xử lý điểm ảnh	3
1.1 Biến đổi mức xám	3
1.2 Biến đổi Log (Log Transformation)	3
1.3 Biến đổi hàm mũ (Power Transform)	4
1.4 Phân ngưỡng	4
1.5 Kỹ thuật Histogram Equalization	4
1.6 Kỹ thuật Histogram Matching	5
Chương 2: Làm việc với không gian màu	6
2.1 Bài tập 1: Biến đổi không gian màu	6
2.2 Bài tập 2: Biến đổi tuyến tính màu	6
2.3 Bài tập 3: Kỹ thuật ChromaKeying	7
2.4 Bài tập 4: Kỹ thuật Shadow Remove	7
Chương 3: Mô hình nhiễu và khử nhiễu	9
3.1 Bài tập 1: Các mô hình nhiễu trong ảnh	9
3.2 Bài tập 2: Bộ lọc tuyến tính	9
3.3 Bài tập 3: Các bộ lọc khác	9
3.4 Bài tập 4: PSNR và SSIM	9
Chương 4: Kỹ thuật xác định biên	11
4.1 Bài tập 1: Đọc ảnh và chuẩn hóa	11
4.2 Bài tập 2: Image Gradient	11
4.3 Bài tập 3: Canny Edge Detector	11
4.4 Bài tập 4: Image Sharpening	11
Chương 5: Giải thuật nén ảnh	12
5.1 Bài tập 1: Thuật toán OMP	12
5.2 Bài tập 2: Nén ảnh chuẩn JPEG	13
Chương 6: Hình thái toán học	15
6.1 Bài tập 1: Xác định biên - hình ảnh CT	15
6.2 Bài tập 2: Khử nhiễu	15
6.3 Bài tập 3: Lấp đầy điểm khuyết thiếu	16
Chương 7: Xác định cạnh	17
7.1 Bài tập 1: Triển khai Hough Transform	17
7.2 Bài tập 2: Xác định cạnh trên ảnh nhiễu	17
7.3 Bài tập 3: Triển khai Hough Circle	17

Chương 8: Thuật toán xác định góc	18
8.1 Bài tập 1: (Thuật toán Harris)	18
8.2 Bài tập 2: Trích xuất đặc trưng	19
8.3 Bài tập 3: Ghép cặp đặc trưng	19
Chương 9: Template matching	20
9.1 Bài tập 1: Tính hệ số tương quan chéo chuẩn hóa	20
9.2 Bài tập 2: Tính bản đồ tương quan	20
9.3 Bài tập 3: Tìm vị trí khớp	20
9.4 Bài tập 4: Áp dụng thuật toán	21
9.5 Bài tập 5: Cải thiện thuật toán	21
9.6 Bài tập 6: Chỉnh sửa hàm <code>find_matched</code> theo yêu cầu sau	21
Chương 10: Biến đổi Fourier 1D	23
10.1 Bài tập 1: biến đổi Fourier 1 chiều	23
10.2 Bài tập 2: Phân tích chuỗi Fourier	23
10.3 Bài tập 3: Triển khai Fourier nhanh	24
Chương 11: Biến đổi Fourier 2D	25
11.1 Bài tập 1 – Bộ lọc thông thấp	25
11.2 Bài tập 2 – Bộ lọc thông cao	25
Deep learning Topic: Convolution Neural Network	27
12.1 Bài tập 1: (Convolution)	27
12.2 Bài tập 2: (Độ đo trong Bài tập classification)	27
12.3 Bài tập 3 *: (Two Layer Net)	28
12.4 Bài tập 4: MNIST (Modified National Institute of Standards and Technology)	29
Deep learning Advance: CNN model	31
13.1 Bài tập 1: (Tập dữ liệu Imagenette2)	31
13.2 Bài tập 2: (Kiến trúc ResNet)	32
13.3 Bài tập 3: (Huấn luyện mô hình)	33

Chương 1: Xử lý điểm ảnh

Mục tiêu: Giúp sinh viên hiểu sâu về cách hình ảnh được biểu diễn và thao tác ở mức cơ bản nhất—từng điểm ảnh.

Yêu cầu thực hành: Tất cả các bài tập trong khóa học thị giác máy tính sẽ được thực hiện trên Google Colab, và ngôn ngữ lập trình sử dụng là Python. Sinh viên cần có kiến thức cơ bản về lập trình Python để hoàn thành khóa học này.

(Kiểm tra kiến thức lập trình) Histogram thể hiện phân bố tần suất của các giá trị cường độ điểm ảnh trong một hình ảnh. Không sử dụng bất kỳ thư viện bên ngoài nào, hãy viết một chương trình tính toán histogram của một ảnh xám 8-bit cho trước. Hiển thị kết quả và so sánh với kết quả được tính toán bằng một thư viện có sẵn dành cho việc tính histogram.

1.1 Biến đổi mức xám

Tạo một hàm `GrayLevelModification` để thay đổi giá trị của ảnh xám 8-bit đầu vào theo công thức:

$$s = 16 \times \sqrt{r}$$

trong đó r là giá trị cường độ đầu vào và s là giá trị đã qua xử lý. Hệ số 16 đảm bảo rằng kết quả nằm trong khoảng từ 0 đến 255. Sử dụng hàm này để biến đổi bất kỳ ảnh xám 8-bit nào hiện có.

1.2 Biến đổi Log (Log Transformation)

Phép biến đổi Logarit giúp giãn nở các khoảng giá trị pixel thấp (vùng tối) và nén các khoảng giá trị pixel cao (vùng sáng), phép biến đổi này được định nghĩa bởi công thức:

$$s = c \cdot \log(1 + r)$$

Trong đó: r là cường độ pixel gốc. s là cường độ pixel sau biến đổi. c là hằng số tỉ lệ, giúp kéo dài giá trị đầu ra về dải hiển thị mong muốn.

Thực hiện các bước sau:

1. Đọc ảnh (**nasa.jpg**) ở chế độ ảnh xám và hiển thị ảnh gốc kèm biểu đồ Histogram. Nhận xét về phân bố mức xám của ảnh.
2. Áp dụng công thức $s = c \cdot \log(1 + r)$ lên ảnh (chuyển ảnh về float để tránh mất thông tin). Với c bất kỳ, hiển thị kết quả lên màn hình.

3. Thực hiện thay đổi hằng số c theo công thức chuẩn hóa:

$$c = \frac{255}{\log(1 + \max(r))} \quad (1.1)$$

4. Chuyển đổi kết quả về kiểu `uint8` và hiển thị so sánh giữa ảnh gốc và ảnh sau biến đổi.
5. Giả sử ta sử dụng phép nhân tuyến tính đơn giản $s = k \cdot r$ (với $k > 1$) để làm sáng ảnh chụp đêm. Tại sao phương pháp tuyến tính này thường dẫn đến kết quả kém hơn so với biến đổi Logarit? Hiện tượng gì sẽ xảy ra tại các vùng vốn dĩ đã sáng trong ảnh gốc?

1.3 Biến đổi hàm mũ (Power Transform)

Tạo một hàm `PowerTransform` thực hiện biến đổi theo luật lũy thừa trên ảnh. Hàm này nhận vào một ảnh xám 8-bit và giá trị gamma (có thể chỉnh sửa trong chương trình) để biến đổi ảnh. Biến đổi theo luật lũy thừa được thực hiện theo công thức:

$$s = c \times r^\gamma$$

trong đó s là giá trị điểm ảnh đã qua xử lý, r là giá trị điểm ảnh gốc, γ là tham số điều chỉnh mức độ biến đổi, và c là một hằng số thường được đặt là 1.

Thử nghiệm hàm này trên các ảnh `spine.jpg` và `runway.jpg`, thay đổi các giá trị γ . Trong phần chú thích của hàm, hãy ghi lại các giá trị γ tốt nhất cho từng ảnh.

1.4 Phân ngưỡng

Tạo một hàm có tên `MedianThreshold` đặt giá trị ngưỡng bằng giá trị trung vị của histogram. Sử dụng hàm này để phân ngưỡng một ảnh xám 8-bit đầu vào.

Định nghĩa: Trung vị m là giá trị thỏa mãn điều kiện:

$$P(x < m) = P(x > m)$$

Nói cách khác, một nửa số giá trị cường độ điểm ảnh nhỏ hơn m và một nửa lớn hơn m .

1.5 Kỹ thuật Histogram Equalization

Tạo một hàm gọi là *HistogramEqualization* nhận đầu vào là hai bức ảnh `source` và `dest`. Thực hiện việc chuyển đổi cường độ pixel của ảnh sao cho phân bố histogram của ảnh đầu ra càng gần với một phân bố đều càng tốt. Đây là một kỹ thuật xử lý ảnh nhằm tăng cường độ tương phản của ảnh bằng cách phân bố lại các mức xám của ảnh sao cho chúng được trải đều hơn trên toàn bộ dải động của ảnh.

Thực hiện theo các bước sau:

1. Tính toán histogram của ảnh gốc.

2. Xây dựng hàm phân phối tích lũy (CDF - Cumulative Distribution Function) từ histogram
3. Ánh xạ mức xám của ảnh gốc sang mức xám mới theo công thức:

$$s_k = (L - 1) \times CDF(r_k) \quad (1.2)$$

4. Tạo ảnh mới bằng cách thay thế mỗi pixel r_k bằng giá trị mới s_k

1.6 Kỹ thuật Histogram Matching

Tạo một hàm gọi là *HistogramMatching* nhận đầu vào là hai ảnh màu *source I* và *dest J*. Mục tiêu là biến đổi cường độ pixel của ảnh *I* để histogram của ảnh kết quả *I'* gần nhất có thể với histogram của ảnh *J*. Ảnh *I'* là ảnh đầu ra sau khi thực hiện biến đổi.

Histogram Matching (hay Histogram Specification) thực hiện "áp" phân bố mức xám/mức cường độ của ảnh nguồn theo phân bố mục tiêu. Với ảnh màu, thao tác được áp dụng độc lập trên từng kênh màu (ví dụ: R, G, B) hoặc trên một kênh độ sáng trong không gian màu phù hợp.

Thực hiện theo các bước sau:

1. Chọn hai ảnh màu *I* (ảnh nguồn) và *J* (ảnh đích).
2. Với mỗi kênh màu $c \in \{R, G, B\}$ (hoặc kênh độ sáng):
 - (a) Tính histogram và chuẩn hoá để thu được phân bố xác suất $p_I^c(r)$ và $p_J^c(z)$.
 - (b) Tính hàm phân phối tích lũy (CDF) cho hai ảnh:

$$CDF_I^c(r) = \sum_{t=0}^r p_I^c(t), \quad CDF_J^c(z) = \sum_{t=0}^z p_J^c(t) \quad (1.3)$$

- (c) Xây dựng ánh xạ mức cường độ từ ảnh nguồn sang ảnh đích theo nguyên tắc: tìm z sao cho $CDF_J^c(z)$ gần nhất với $CDF_I^c(r)$:

$$T^c(r) = \arg \min_{z \in \{0, \dots, L-1\}} |CDF_I^c(r) - CDF_J^c(z)| \quad (1.4)$$

trong đó L là số mức cường độ (thường $L = 256$ với ảnh 8-bit).

- (d) Tạo kênh ảnh mới bằng cách thay thế mỗi pixel có giá trị r bởi $T^c(r)$.
3. Ghép các kênh sau biến đổi để thu được ảnh kết quả *I'* và lưu ảnh *I'*.

2.1 Bài tập 1: Biến đổi không gian màu

Cho ảnh màu $I \in \mathbb{R}^{m \times n \times 3}$ (lenna_color.jpg). Yêu cầu:

1. Đọc và hiển thị ảnh I trong không gian màu RGB.
2. Trích xuất và hiển thị riêng biệt ba kênh màu R, G, B.
3. Chuyển đổi ảnh từ không gian màu RGB sang không gian màu HSV.
4. Hiển thị riêng biệt ba kênh H, S, V của ảnh HSV.

Tiếp theo, với ảnh đã được chuyển đổi sang không gian màu HSV:

1. Đọc ảnh I' trong không gian màu HSV và hiển thị các kênh H, S, V.
2. Thực hiện chuyển đổi ảnh từ không gian màu HSV ngược lại không gian màu RGB.
3. So sánh ảnh thu được với ảnh gốc I , đánh giá sự thay đổi về màu sắc và độ chính xác của phép biến đổi.

2.2 Bài tập 2: Biến đổi tuyến tính màu

Trong xử lý ảnh, phép **biến đổi tuyến tính màu** được sử dụng để hiệu chỉnh màu sắc của ảnh, giúp đảm bảo màu sắc hiển thị chính xác hơn hoặc thay đổi phong cách màu của ảnh. Phép biến đổi này được biểu diễn dưới dạng ma trận A có kích thước 3×3 , ánh xạ màu từ ảnh đầu vào J (ảnh sai màu) sang ảnh gốc I (ảnh chuẩn). Phép biến đổi này được mô tả bởi phương trình:

$$Y = A^{-1}X \quad (2.1)$$

Với:

- X là ma trận $3 \times n$ chứa n điểm màu từ ảnh sai màu J .
- Y là ma trận $3 \times n$ chứa n điểm màu từ ảnh gốc I .
- A^{-1} là ma trận hiệu chỉnh màu cần tìm.

Yêu cầu thực hiện

1. Đọc và hiển thị hai ảnh I (images.jpg) và J (images2.jpg).

2. **Chọn ít nhất 3 điểm màu** từ ảnh I và ảnh J (các điểm tương ứng).
 - Các điểm này nên phân bố đều trong không gian màu, bao gồm vùng sáng, vùng tối và vùng trung tính.
 - Ghi lại giá trị RGB của các điểm màu tương ứng.
3. **Tính toán ma trận hiệu chỉnh màu** A bằng cách giải hệ phương trình tuyến tính:

$$B = A^{-1} = YX^{-1} \quad (2.2)$$
4. **Áp dụng ma trận** A lên toàn bộ ảnh J để tạo ra ảnh đã hiệu chỉnh.
5. **Hiển thị và so sánh ảnh** sau khi hiệu chỉnh với ảnh gốc.
6. **(Mở rộng)** Tăng số điểm mapping lên $n > 3$ và sử dụng phương pháp *Least Squares* để tìm ma trận A tối ưu hơn:

$$B = A^{-1} = YX^T(XX^T)^{-1} \quad (2.3)$$

2.3 Bài tập 3: Kỹ thuật ChromaKeying

Cho ảnh $I \in \mathbb{R}^{m \times n \times 3}$ (lion.jpg) và ảnh $J \in \mathbb{R}^{m \times n \times 3}$ (lionHSV.jpg). Thực hiện yêu cầu sau:

1. Đọc và hiển thị ảnh I, J lên cùng một khung hình.
2. Gọi 3 kênh màu tương ứng của J là J_h, J_s và J_v . Chuyển kiểu của J_h về kiểu `double`. Nhị phân hoá J_h bằng cách đặt những giá trị lớn hơn 0.22 và nhỏ hơn 0.45 bằng 1, các giá trị khác bằng 0. Gọi ảnh nhị phân thu được là B , hiển thị ảnh B lên màn hình.
3. Sử dụng đoạn mã giả sau nhằm xử lý thông nền màu xanh trên ảnh $I \in \mathbb{R}^{m \times n \times 3}$. Trong đó cột số 1, 2, 3, 4 trong tệp `data.txt` chứa thông tin về các vector v_1^T, v_2^T, v_3^T , và v_B .
Hiển thị ảnh I, K trên cùng một khung hình.

2.4 Bài tập 4: Kỹ thuật Shadow Remove

Thực hiện các yêu cầu sau:

1. Cho ảnh $I(images.jpg)$ và $J(binary.png) \in \mathbb{R}^{h \times w}$. Biến đổi ảnh I sao cho I và J có cùng kích thước. Chuyển kiểu của I về `double`. Gọi $\mathbf{dbI} = I / 255$. Hiển thị $\mathbf{dbI}, J$ lên màn hình dưới mỗi ảnh là thông tin về kích thước của ảnh đó.

ảnh I	ảnh J
kích thước của ảnh I	kích thước của ảnh J

Algorithm 1 CHROMAKEYING

```
1: procedure CHROMAKEYING
2:   Tính  $id$  là vector cột chứa chỉ số của các phần tử khác 0 trong  $v_B$ 
3:   for  $b = 1$  to 3 do
4:     Gán các phần tử trong  $v_T^b$  có chỉ số  $id$  giá trị bằng 0
5:      $K_b = \text{zeros}(m, n)$ 
6:     for  $j = 1$  to  $n$  do
7:       count = 1
8:       for  $i = (j - 1) \times m + 1$  to  $j \times m$  do
9:          $K_b(\text{count}, j) = v_T^b[i]$ 
10:        count = count + 1
11:      end for
12:    end for
13:  end for
14:  Kết hợp 3 kênh màu  $K_b$  ( $b = 1, 2, 3$ ) để tạo thành ảnh màu  $K$ 
15:  Chuyển kiểu của  $K$  về kiểu  $\text{uint8}$ 
16: end procedure
```

2. (a) Nhị phân hoá ma trận J sao cho pixel trắng có giá trị bằng 0 và pixel màu đen có giá trị bằng 1.
(b) Đọc dữ liệu từ file **alpha.txt** và hiển thị hình ảnh.
3. Tìm ảnh SF sử dụng thuật toán sau. Hiển thị SF và ảnh I lên màn hình.

Algorithm 2 Thuật toán Shadow Remove

```
1: procedure SHADOWREMOVE
2:    $\mathbf{r} = \begin{pmatrix} 1.2 \\ 1.0 \\ 0.7 \end{pmatrix}$ 
3:    $[m, n] = \text{size}(\alpha)$ 
4:    $\mathbf{I} \in \mathbb{R}^{m \times n}$  là ma trận toàn giá trị 1
5:   for  $i = 1$  to 3 do
6:      $c^{(i)}$  là kênh màu thứ  $i$  của ảnh  $\text{dbl}$ 
7:      $rc^{(i)} = \mathbf{I} \times \frac{r^{(i)} + 1}{\alpha \times r^{(i)} + 1} \cdot c^{(i)}$ 
8:   end for
9:   Kết hợp 3 kênh màu  $rc^{(i)}$  để thu được ảnh SF
10:  Đặt giá trị SF > 1 bằng 1
11: end procedure
```

Trong đó, phép toán $\cdot$ và $\cdot$ giữa hai ma trận $A = (a_{ij})_{m \times n}$, $B = (b_{ij})_{m \times n}$ được định nghĩa như sau:

$$C = A \cdot B = (a_{ij}/b_{ij})_{m \times n}, \quad D = A \cdot B = (a_{ij} \times b_{ij})_{m \times n}$$

3.1 Bài tập 1: Các mô hình nhiễu trong ảnh

Sử dụng hai ảnh `barbara.png` và `pepper.png`, anh/chị hãy tạo ảnh nhiễu cho các ảnh trên, sử dụng các mô hình nhiễu sau:

- Nhiễu Salt and Pepper
- Nhiễu Gaussian
- Nhiễu Speckle

3.2 Bài tập 2: Bộ lọc tuyến tính

Hãy sử dụng bộ lọc tuyến tính với kích thước nhân 3×3 và 7×7 để giảm nhiễu các bức ảnh ở trên. Anh/chị có nhận xét gì về ảnh sau khi khử nhiễu?

3.3 Bài tập 3: Các bộ lọc khác

Hãy sử dụng các bộ lọc phi tuyến sau để khử nhiễu ảnh:

- Median filter
- Weight median filter
- Outlier method

Anh/chị có nhận xét gì về ảnh giảm nhiễu bằng các phương pháp trên?

3.4 Bài tập 4: PSNR và SSIM

Để đánh giá chất lượng của ảnh giảm nhiễu, các chỉ số *Peak Signal-to-Noise Ratio* (PSNR) và *Structural Similarity Index Measure* (SSIM) là các chỉ số thường được dùng. Hai chỉ số này được tính như sau:

PSNR

$$\text{PSNR} = 10 \log_{10} \left(\frac{R^2}{\text{MSE}} \right) \quad (3.1)$$

với MSE là sai số toàn phương trung bình giữa ảnh giảm nhiễu và ảnh gốc, R là độ rộng khoảng giá trị của ảnh.

SSIM

SSIM được tính trên từng miếng vá nhỏ của ảnh, sau đó lấy giá trị trung bình. Giá trị SSIM giữa hai miếng vá x và y có kích thước $N \times N$ được tính như sau:

$$\text{SSIM} = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)} \quad (3.2)$$

với:

- μ_x, μ_y lần lượt là giá trị trung bình của x và y
- σ_x^2, σ_y^2 lần lượt là phương sai của x và y
- σ_{xy} là hiệp phương sai của x và y
- c_1 và c_2 là hai biến để ổn định phép chia khi mẫu số yếu. Thông thường, ta chọn $c_1 = (0.01R)^2, c_2 = (0.03R)^2$, với R là độ rộng khoảng giá trị của ảnh.

Anh/chị hãy viết hàm tính hai chỉ số trên, sau đó áp dụng hàm để tính các chỉ số này cho ảnh giảm nhiễu ở Câu 2 và Câu 3.

Chương 4: Kỹ thuật xác định biên

Cho ảnh `lenna.png`. Thực hiện các yêu cầu sau đây:

4.1 Bài tập 1: Đọc ảnh và chuẩn hóa

- Đọc ảnh `lenna.png` vào máy dưới dạng ảnh đen trắng (grayscale).
- Đưa giá trị của ảnh về khoảng $[0, 1]$ bằng cách chuẩn hóa.

4.2 Bài tập 2: Image Gradient

1. Tính gradient của ảnh theo hai chiều x và y bằng cách sử dụng các bộ lọc Sobel, Prewitt và Robert.
2. Tính **hướng** và **độ lớn** của vector gradient tại mỗi điểm ảnh. Gọi I là ảnh biểu diễn độ lớn gradient.
3. Hiển thị ảnh I lên màn hình.
4. Chọn giá trị ngưỡng phù hợp, thực hiện lọc ảnh I theo quy tắc:
 - Nếu giá trị độ lớn lớn hơn ngưỡng, đặt giá trị bằng 1.
 - Ngược lại, đặt giá trị bằng 0.
5. Hiển thị ảnh sau khi lọc ngưỡng lên màn hình.

4.3 Bài tập 3: Canny Edge Detector

- Giảm nhiễu ảnh bằng bộ lọc Gaussian kích thước 5×5 .
- Sử dụng hàm `cv2.Canny()` trong thư viện OpenCV để thực hiện thuật toán phát hiện biên Canny.
- Hiển thị kết quả thu được.
- So sánh kết quả thu được từ bài tập 2 và bài tập 3.

4.4 Bài tập 4: Image Sharpening

1. Làm mờ ảnh gốc với bộ lọc trung bình kích thước 5×5 .
2. Áp dụng thuật toán *image sharpening* (làm sắc nét ảnh) lên ảnh đã làm mờ.
3. Đưa ra nhận xét về kết quả thu được.

Chương 5: Giải thuật nén ảnh

5.1 Bài tập 1: Thuật toán OMP

Đọc ảnh đầu vào I (*lenna.png*) mode Gray-scale và thực hiện các yêu cầu sau:

- Viết hàm *blockDivide(img, block_size)* thực hiện việc chia ảnh thành các block kích thước *block_size* sau đó là phẳng khối block. Kết quả trả về làm một mảng vector cột, là thông tin vector block.
- Sinh từ điển dictionary D như sau:

Algorithm 3 Tính toán ma trận DCT 1D

```

1: function DCT_1D(N)
2:   Khởi tạo ma trận dct_mat kích thước  $N \times N$  với giá trị 0
3:   for  $k = 0$  to  $N - 1$  do
4:     for  $n = 0$  to  $N - 1$  do
5:       if  $k = 0$  then
6:          $dct\_mat[k, n] = \frac{1}{\sqrt{N}}$ 
7:       else
8:          $dct\_mat[k, n] = \sqrt{\frac{2}{N}} \times \cos\left(\frac{\pi(2n+1)k}{2N}\right)$ 
9:       end if
10:    end for
11:  end for
12:  return dct_mat
13: end function

```

Algorithm 4 Tính toán ma trận cơ sở DCT 2D

```

1: function DCT2_BASIS(B)
2:    $dct1 \leftarrow \text{DCT\_1D}(B)$  ▷ Lấy ma trận DCT 1D kích thước  $B \times B$ 
3:    $dct2d \leftarrow$  Kronecker product of dct1 and dct1
4:   return dct2d
5: end function

```

- Trong Bài tập nén ảnh, ta có thể biểu diễn ảnh dưới dạng tổ hợp tuyến tính của một số hữu hạn ít các atoms trong một từ điển (dictionary). Quá trình này thực hiện trên từng khối block đã tạo được ở trên và được mô tả như sau:
- Thực hiện tái tạo lại ảnh. Điều chỉnh thông số *sparsity*, tính toán chỉ số PSNR và SSIM trên ảnh đã tái tạo và ảnh ban đầu.

Algorithm 5 Orthogonal Matching Pursuit (OMP)

Require: D : Ma trận từ điển kích thước (M, K)
Require: x : Vector ảnh (hoặc block) cần biểu diễn, kích thước $(M,)$
Require: $sparsity$: Số lượng phần tử khác 0 tối đa
Ensure: α : Vector trọng số thừa kích thước $(K,)$

- 1: Khởi tạo $residual \leftarrow x$
- 2: Khởi tạo danh sách $used_atoms \leftarrow \emptyset$
- 3: Khởi tạo $\alpha \leftarrow$ vector 0 kích thước $(K,)$
- 4: **for** $i = 1$ **to** $sparsity$ **do**
- 5: **Bước 1:** Chọn atom có tương quan lớn nhất
- 6: $correlations \leftarrow D^T \cdot residual$
- 7: $atom_index \leftarrow \arg \max |correlations|$
- 8: Thêm $atom_index$ vào danh sách $used_atoms$
- 9: **Bước 2:** Giải Bài tập Least Squares trên các atom đã chọn
- 10: $D_used \leftarrow D[:, used_atoms]$
- 11: $\alpha_used \leftarrow \arg \min \|x - D_used \cdot \alpha_used\|_2$
- 12: **Bước 3:** Cập nhật residual
- 13: $residual \leftarrow x - D_used \cdot \alpha_used$
- 14: **Bước 4:** Kiểm tra dừng sớm
- 15: **if** $\|residual\|_2 < 10^{-6}$ **then**
- 16: **Break**
- 17: **end if**
- 18: **end for**
- 19: **Bước 5:** Cập nhật lại α
- 20: **for** $j = 1$ **to** số lượng phần tử trong $used_atoms$ **do**
- 21: $\alpha[used_atoms[j]] \leftarrow \alpha_used[j]$
- 22: **end for**
- 23: **return** α

5.2 Bài tập 2: Nén ảnh chuẩn JPEG

Đọc ảnh I (*lenna.png*) và thực hiện yêu cầu sau:

1. Viết hàm *blockDivide(img, block_size)* thực hiện việc chia ảnh thành các block kích thước 8×8 sau đó là phẳng khối block. Kết quả trả về làm một mảng vector cột, là thông tin vector block.
2. **Tiền xử lý:** Tạo ra miền không gian có giá trị trung bình ≈ 0 bằng cách trừ giá trị mỗi pixel trong block đi 128. Sau đó, mỗi block (8×8) được chuyển sang miền tần số thông qua biến đổi DCT (tham khảo 4).
3. Khởi tạo ma trận lượng tử hoá chuẩn JPEG (ma trận Q), các hệ số DCT được lượng tử hoá bằng cách chia cho ma trận đã tạo (làm tròn).

$$Q = \begin{bmatrix} 16 & 11 & 10 & 16 & 24 & 40 & 51 & 61 \\ 12 & 12 & 14 & 19 & 26 & 58 & 60 & 55 \\ 14 & 13 & 16 & 24 & 40 & 57 & 69 & 56 \\ 14 & 17 & 22 & 29 & 51 & 87 & 80 & 62 \\ 18 & 22 & 37 & 56 & 68 & 109 & 103 & 77 \\ 24 & 35 & 55 & 64 & 81 & 104 & 113 & 92 \\ 49 & 64 & 78 & 87 & 103 & 121 & 120 & 101 \\ 72 & 92 & 95 & 98 & 112 & 100 & 103 & 99 \end{bmatrix}$$

4. Khi giải nén, ta nhân lại với ma trận lượng tử hóa rồi áp dụng IDCT để khôi phục block (*lưu ý khối block này cần phải được trả về giá trị ban đầu bằng cách cộng thêm 128*).

$$f(x, y) = \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} \alpha(u) \alpha(v) F(u, v) \cos\left(\frac{\pi(2x+1)u}{2N}\right) \cos\left(\frac{\pi(2y+1)v}{2N}\right)$$

$$\alpha(u) = \begin{cases} \sqrt{\frac{1}{N}}, & \text{nếu } u = 0, \\ \sqrt{\frac{2}{N}}, & \text{nếu } u > 0, \end{cases} \quad \alpha(v) = \begin{cases} \sqrt{\frac{1}{N}}, & \text{nếu } v = 0, \\ \sqrt{\frac{2}{N}}, & \text{nếu } v > 0. \end{cases}$$

trong python có thể sử dụng nhanh chóng thông qua thư viện theo cú pháp: from scipy.fftpack import dct, idct

5. Tính toán chỉ PSNR và SSIM trên ảnh gốc và ảnh tái tạo. Kiểm tra dung lượng 2 ảnh, cho nhận xét.

6.1 Bài tập 1: Xác định biên - hình ảnh CT

1. Đọc và hiển thị hình ảnh `brainCT.png`.
2. Kiểm tra biểu đồ tần suất (histogram) của hình ảnh và thử tìm một ngưỡng T có thể tách xương khỏi nền. Áp dụng ngưỡng T , sẽ thấy rằng không thể tìm một ngưỡng hoàn hảo. Do đó, bài tập tiếp theo là đóng các lỗ trong xương trong khi vẫn giữ được hình dạng tổng thể.
3. Thử nghiệm với các kết hợp khác nhau của phép mở (open) và phép đóng (close) với các kích thước kernel khác nhau trên ảnh nhị phân từ bước 2. Có thể thử các tổ hợp trong đó kernel sử dụng cho phép mở khác với kernel sử dụng cho phép đóng. Gọi `Imask` là kết quả tốt nhất của bạn.
4. Xác định biên của xương bằng cách sử dụng phép dẫn nở như sau và hiển thị hình ảnh ranh giới:
 - Dilate `Imask` với một kernel nhỏ.
 - Trừ `Imask` khỏi hình ảnh đã dẫn nở.
5. Xác định biên của xương bằng cách sử dụng phép Erode. Hiển thị hình ảnh ranh giới mới và so sánh với hình ảnh ở bước 4. Giải thích sự khác biệt giữa các hình ảnh liên quan đến các toán tử hình thái học được sử dụng.
 - Erode `Imask` với một kernel nhỏ.
 - Trừ hình ảnh đã Erode khỏi `Imask`.

6.2 Bài tập 2: Khử nhiễu

1. Đọc và hiển thị hình ảnh `images2.jpg` - Figure 1.
2. Thay đổi kích thước ảnh về 224x224.
3. Kiểm tra biểu đồ tần suất (histogram) của hình ảnh và thử tìm một ngưỡng T nhị phân hóa ảnh, gọi là ảnh `A`.
4. Khử nhiễu ảnh theo các bước sau:
 - Khởi tạo SE kích thước 3x3 toàn giá trị 1 gọi là `B`.



Hình 1: Ảnh đầu vào bị nhiễu

- mỗi bước tiến hành hiển thị kết quả theo thứ tự như sau:

$$A \ominus B \rightarrow (A \ominus B) \oplus B = A \circ B \rightarrow (A \circ B) \oplus B \rightarrow [(A \circ B) \oplus B] \ominus B = (A \circ B) \bullet B$$

5. Áp dụng bộ lọc trung vị khử nhiễu trên ảnh nhiễu ban đầu, so sánh và nhận xét 2 kết quả.

6.3 Bài tập 3: Lấp đầy điểm khuyết thiếu

1. Đọc và hiển thị hình ảnh `hole.png`.
2. Kiểm tra biểu đồ tần suất (histogram) của hình ảnh và thử tìm một ngưỡng T nhị phân hóa ảnh, gọi là ảnh J.
3. Thực hiện lấp phần trống trong đối tượng theo yêu cầu sau:
 - Đảo bit ảnh J, lúc này đối tượng là màu đen, background là màu trắng.
 - Xác định một ảnh đánh dấu (marker) ban đầu là các điểm nằm ở biên ảnh (vì những điểm này chắc chắn là nền thật, không phải hole).
 - Dùng phép giãn (dilation) - dạng ELLIPSE, trên vùng marker sẽ cố gắng tô hết ảnh và coi đó là background (màu trắng). Giao của ảnh J và marker chính là vùng thực sự là background (màu trắng).
 - Khi quá trình lặp hội tụ, tức là giao của J và marker khi dần nở không thay đổi (phần hole đã được tô hết).
 - Cuối cùng, đảo ngược lại để được ảnh đã lấp lỗ.

7.1 Bài tập 1: Triển khai Hough Transform

Viết hàm thực hiện các công việc sau:

1. Áp dụng bộ phát hiện cạnh Canny để xây dựng bản đồ cạnh cho ảnh "ames1.JPG".
2. Sử dụng Hough Transform để xác định các đường thẳng trong ảnh. Vẽ các đường thẳng xác định được lên ảnh ban đầu.
3. Hiển thị ảnh ban đầu, bản đồ cạnh và ảnh sau khi xác định các cạnh.

7.2 Bài tập 2: Xác định cạnh trên ảnh nhiễu

Cho ảnh "t_img.png". Thực hiện các thao tác sau với ảnh:

- Thực hiện thêm nhiễu Gauss lên ảnh trên.
- Áp dụng thuật toán Hough Transform để xác định các cạnh trong ảnh.
- Xác định xem với mức nhiễu Gauss bao nhiêu được thêm vào ảnh, thuật toán Hough Transform không còn khả năng xác định các đường thẳng trong ảnh?

7.3 Bài tập 3: Triển khai Hough Circle

Cho ảnh đầu vào "coin.jpg". Thực hiện thao tác sau:

1. Hiển thị ảnh lên màn hình.
2. Chuyển ảnh sang kênh màu xám và áp dụng bộ lọc khử nhiễu Gauss lên ảnh. Hiển thị kết quả.
3. Triển khai HoughCircle xác định đường tròn trên ảnh và chọn tham số thích hợp.
4. Hiển thị bản đồ cạnh và tâm sau khi xác định hình tròn.

Chương 8: Thuật toán xác định góc

Giảng viên: Đỗ Thanh Hà

VNU - University of Science

8.1 Bài tập 1: (Thuật toán Harris)

Viết một hàm `corner(image: np.ndarray, w: int, kappa: float)` nhận vào ảnh `image_1.png`, `image_2.png` và thực hiện phát hiện góc theo thuật toán Harris.

1. Áp dụng bộ lọc Gaussian kích thước $w \times w$ lên ảnh I để khử nhiễu.
2. Tính gradient I_x, I_y của ảnh theo hai chiều x và y .
3. Tính toán toán tử Harris tại mỗi điểm ảnh (x, y) như sau:

$$K(x, y) = \det(H(x, y)) - \kappa \times \text{trace}^2(H(x, y)) \quad (8.1)$$

Với:

$$\det(H(x, y)) = I_x^2 I_y^2 - (I_x I_y)^2 \quad (8.2)$$

$$\text{trace}(H(x, y)) = I_x^2 + I_y^2 \quad (8.3)$$

Và ma trận Hessian $H(x, y)$ tại điểm (x, y) được cho bởi:

$$H(x, y) = \begin{bmatrix} \sum_{(u,v) \in W(x,y)} w(u, v) I_x^2(x+u, y+v) & \sum_{(u,v) \in W(x,y)} w(u, v) I_x I_y(x+u, y+v) \\ \sum_{(u,v) \in W(x,y)} w(u, v) I_x I_y(x+u, y+v) & \sum_{(u,v) \in W(x,y)} w(u, v) I_y^2(x+u, y+v) \end{bmatrix}$$

Trong đó:

- I_x và I_y là gradient của ảnh theo hướng x và y tương ứng,
 - $W(x, y)$ là cửa sổ lân cận xung quanh điểm ảnh (x, y) ,
 - $w(u, v)$ là trọng số tại vị trí tương đối (u, v) trong cửa sổ (thường là Gaussian).
4. Tìm tập hợp các điểm ảnh $H = \{(x, y) : K(x, y) > \kappa\}$, trong đó phản hồi của toán tử Harris lớn hơn ngưỡng κ .
 5. Áp dụng kỹ thuật **non-maximum suppression** trong một vùng lân cận $w \times w$ xung quanh mỗi điểm $(x, y) \in H$. Lưu lại tọa độ các điểm ảnh được chọn làm keypoints.

8.2 Bài tập 2: Trích xuất đặc trưng

Viết một hàm `features(image: np.ndarray, keypoints: np.ndarray)` để trích xuất descriptor xung quanh mỗi điểm keypoint trong `keypoints`. Sử dụng patch ảnh kích thước 9×9 được vector hóa làm descriptor đơn giản.

8.3 Bài tập 3: Ghép cặp đặc trưng

Viết hàm `matching(descriptors1: np.ndarray, descriptors2: np.ndarray, tau: float)` để ghép hai tập các descriptor bằng cách sử dụng **tổng bình phương sai khác** (SSD) làm thước đo. Hàm của bạn cần trả về:

- Với mỗi descriptor trong ảnh 1, trả về chỉ số của descriptor tốt nhất khớp trong ảnh 2, nếu điểm SSD nhỏ hơn ngưỡng τ .
- Nếu một descriptor trong ảnh 1 không có khớp nào phù hợp, gán chỉ số tương ứng bằng -1 .

Viết một đoạn script để kiểm tra các hàm của bạn trên cặp ảnh `"image_1.png"` và `"image_2.png"`, và vẽ các kết quả ghép tương ứng.

Chương 9: Template matching

Giới thiệu bài toán

Cho ba bức ảnh: `barton2.jpg`, `barton2-small.jpg` và `barton2-window.jpg`. Thực hiện các bài tập sau:

9.1 Bài tập 1: Tính hệ số tương quan chéo chuẩn hóa

Viết hàm `correlation(img1: np.ndarray, img2: np.ndarray)` thực hiện các thao tác sau:

- Chuyển ảnh `img1` và `img2` thành các vectơ 1 chiều.
- Tính hệ số tương quan chéo chuẩn hóa giữa hai ảnh theo công thức:

$$\frac{1}{n} \sum_{x,y} \frac{1}{\sigma_1 \sigma_2} (f(x,y) - \mu_f)(g(x,y) - \mu_g)$$

trong đó:

- n : tổng số pixel của ảnh,
- $f(x,y)$: giá trị điểm ảnh tại vị trí (x,y) của `img1`,
- μ_f, σ_f : giá trị trung bình và độ lệch chuẩn của ảnh `img1`,
- $g(x,y), \mu_g, \sigma_g$: là các giá trị tương ứng của ảnh `img2`.

9.2 Bài tập 2: Tính bản đồ tương quan

Viết hàm `get_correlation_map(img: np.ndarray, template: np.ndarray)` để tính giá trị tương quan chéo của từng miếng vá (patch) trong ảnh `img` có kích thước bằng ảnh `template` (không cần padding ảnh `img`).

9.3 Bài tập 3: Tìm vị trí khớp

Viết hàm `find_matched(image: np.ndarray, corr_map: np.ndarray, template_size: tuple[int, int], threshold: float)`:

- Tìm các vị trí trong ảnh mà giá trị tương quan lớn hơn ngưỡng `threshold`.
- Đánh dấu các vị trí tìm được bằng cách vẽ hình chữ nhật có kích thước `template_size`.

9.4 Bài tập 4: Áp dụng thuật toán

Viết hàm `main()` thực hiện:

- Đọc ảnh `barton2-small.jpg` và `barton2-window.jpg`.
- Chuyển cả hai ảnh sang thang xám.
- Sử dụng ảnh `barton2-window.jpg` làm template để xác định các vùng trong ảnh `barton2-small.jpg` giống với template.
- Lặp lại với ảnh `barton2.jpg`.

9.5 Bài tập 5: Cải thiện thuật toán

Từ kết quả ở Bài tập 4, có thể thấy một số cửa sổ chưa được phát hiện. Hãy đề xuất phương án cải thiện thuật toán.

Gợi ý: Kết hợp thêm các thuật toán phát hiện góc (corner detection) hoặc phát hiện biên (edge detection) để hỗ trợ.

9.6 Bài tập 6: Chỉnh sửa hàm `find_matched` theo yêu cầu sau

Khi hiển thị ảnh sẽ quan sát thấy một vài vị trí có box bị chồng lấn lên nhau. Thực hiện yêu cầu sau để khắc phục điều này:

- Viết hàm tính toán IoU giữa hai box `IoU(box1, box2)` với mỗi box được biểu diễn bằng 2 tọa độ (x_1, y_1) , (x_2, y_2) . Thực hiện tính IoU theo công thức sau:

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|}$$

Trong đó:

- $A \cap B$: diện tích phần giao nhau của 2 box,
 - $A \cup B$: diện tích phần hợp của 2 box.
- Viết hàm `non_max_suppression(bboxes, scores, iou_threshold)`, trong đó:
 - `bboxes`: danh sách các tọa độ matching thu được từ `corr_map` (chính là các box vẽ được ở bài luyện tập 1),
 - `scores`: mảng chứa độ tương quan (correlation) tương ứng với mỗi box,
 - `iou_threshold`: ngưỡng IoU cho phép hai box được trùng lặp.

Gợi ý: Quá trình loại bỏ các box sẽ bắt đầu từ những box có điểm tương quan (corr) cao nhất. Cần sắp xếp lại các box theo điểm `score`. Những box nào có độ chồng lấn lớn hơn `iou_threshold` sẽ bị loại bỏ.

- Hoàn thiện lại hàm với tên gọi `nms_find_matched()`
`image: np.ndarray,`
`corr_map: np.ndarray,`
`template_size: tuple[int, int],`
`threshold: float,`
`iou_threshold: float = 0.5)`

Chương 10: Biến đổi Fourier 1D

10.1 Bài tập 1: biến đổi Fourier 1 chiều

a) **Viết hàm `dft(x: np.ndarray)`**

Cài đặt Biến đổi Fourier rời rạc (DFT) theo công thức

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i2\pi kn/N} = \sum_{n=0}^{N-1} x_n [\cos(2\pi kn/N) - i \sin(2\pi kn/N)]. \quad (10.1)$$

b) **Viết hàm `idft(x: np.ndarray)`**

Cài đặt Biến đổi Fourier ngược (IDFT)

$$x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k e^{i2\pi kn/N} = \frac{1}{N} \sum_{k=0}^{N-1} X_k [\cos(2\pi kn/N) + i \sin(2\pi kn/N)]. \quad (10.2)$$

c) **Tổng hợp tín hiệu điều hoà**

Sinh ba dao động

$$\begin{cases} x_1(t) = 3 \sin(2\pi \cdot 1 \text{ Hz } t), \\ x_2(t) = 1 \sin(2\pi \cdot 4 \text{ Hz } t), \\ x_3(t) = 0.5 \sin(2\pi \cdot 7 \text{ Hz } t), \end{cases}$$

rồi cộng lại $x(t) = x_1(t) + x_2(t) + x_3(t)$. Lấy mẫu với tần số $f_s = 100$ Hz và vẽ đồ thị đường của $x(t)$.

d) **Phân tích phổ tần số**

Thực hiện DFT cho tín hiệu ở mục (c), chuẩn hoá trục tần số bằng $f = k \frac{f_s}{N}$ ($k = 0 \dots N/2$) và vẽ biên độ phổ. *Nhận xét*: phổ thu được phải có ba đỉnh rõ tại 1 Hz, 4 Hz, 7 Hz—ứng với ba thành phần đã tổng hợp.

10.2 Bài tập 2: Phân tích chuỗi Fourier

Cho tệp dữ liệu `ex2-data.npy` (tần số lấy mẫu $f_s = 100$ Hz).

a) Đọc dữ liệu, tính DFT, vẽ (i) tín hiệu gốc và (ii) biên độ phổ.

- b) Chọn ngưỡng threshold thích hợp, đặt biên độ $< \text{threshold}$ bằng 0, rồi IDFT để tái tạo tín hiệu đã lọc. Vẽ tín hiệu sau lọc và phổ tương ứng.
- c) **Nhận xét:** so sánh nhiễu/độ mượt của tín hiệu trước–sau lọc và sự suy giảm thành phần tần số không mong muốn.

10.3 Bài tập 3: Triển khai Fourier nhanh

Hãy viết các hàm `fft(x: np.ndarray)` và `ifft(x: np.ndarray)` thực hiện biến đổi Fourier và biến đổi Fourier ngược, sử dụng thuật toán biến đổi Fourier nhanh.:

- `fft(x: np.ndarray)` – đệ quy (hoặc lặp) theo sơ đồ *Cooley–Tukey*, độ phức tạp $\mathcal{O}(N \log N)$.
- `ifft(x: np.ndarray)` – tận dụng tính chất $\text{IFFT} = \frac{1}{N} \overline{\text{FFT}(\overline{x})}$.

Gợi ý triển khai nhanh:

Listing 1: Khung mã Python tối giản cho FFT/ IFFT

```
def fft(x):
    N = x.size
    if N == 1:
        # i u k i n d n g
        return x
    X_even = fft(x[::2])
    X_odd = fft(x[1::2])
    factor = np.exp(-2j*np.pi*np.arange(N)//N)
    return np.concatenate([X_even + factor[:N//2]*X_odd,
                           X_even + factor[N//2:]*X_odd])

def ifft(X):
    return np.conjugate(fft(np.conjugate(X))) / X.size
```

11.1 Bài tập 1 – Bộ lọc thông thấp

- a) Viết hàm `low_pass_filter(image: np.ndarray, threshold: float)` gồm các bước:
1. Thực hiện biến đổi Fourier 2-chiều đối với ảnh `image`.
 2. Loại bỏ (đặt bằng 0) các thành phần tần số có biên độ *lớn hơn* `threshold`.
 3. Áp dụng biến đổi Fourier ngược lên phổ đã lọc.
 4. Trả về (i) ảnh sau lọc và (ii) phổ Fourier của ảnh sau lọc.
- b) Áp dụng `low_pass_filter` cho ảnh `barbara.png`. Hiển thị phần *thực* và *ảo* của ảnh kết quả, rồi nhận xét.

11.2 Bài tập 2 – Bộ lọc thông cao

- a) Viết hàm `high_pass_filter(image: np.ndarray, threshold: float)` thực hiện:
1. DFT 2-chiều của `image`.
 2. Loại bỏ các tần số có biên độ *nhỏ hơn* `threshold`.
 3. IDFT để thu được ảnh đã lọc.
 4. Trả về ảnh sau lọc và phổ tương ứng.
- b) Áp dụng hàm cho ảnh `barbara-gaussian.png`, hiển thị phần thực/ảo và nêu nhận xét.

Bài tập 3 – Phát hiện độ mờ (Blur Detection)

Viết hàm `blur_detection(image: np.ndarray, size: int, threshold: float)` theo quy trình:

1. Thực hiện DFT 2-chiều của ảnh `image`.
2. Dịch chuyển thành phần DC vào trung tâm (`np.fft.fftshift`).
3. Tiến hành loại bỏ khu vực trung tâm của ảnh thu được ở bước 2, với kích thước vùng loại bỏ là $2\text{size} \times 2\text{size}$.

4. Thực hiện các biến đổi để tái tạo lại ảnh sau khi xóa bỏ khu vực trung tâm. Gọi ảnh trên là ảnh I (ảnh gồm các giá trị phức).

5. Tính *magnitude-map*

$$M(x, y) = 20 \cdot \log(|I(x, y)|).$$

Lấy trung bình $\overline{M}$.

6. Nếu $\overline{M} < \text{threshold}$ kết luận ảnh *bị mờ*, ngược lại *rõ*.

- Áp dụng cho `barbara.png`.
- Sau đó làm mờ ảnh bằng bộ lọc trung bình $3 \times 3, 5 \times 5, \dots, 11 \times 11$ và chạy lại `blur_detection`; so sánh kết quả.

Bài tập 4 – Template Matching bằng Fourier

1. Chuyển `image` và `template` về ảnh xám.
2. Lấy DFT hai ảnh; `resize/pad template` để kích thước khớp `image`.
3. Tính

$$C = \mathcal{F}^{-1} \left(\frac{\mathcal{F}(I) \circ \mathcal{F}(J)}{|\mathcal{F}(I) \circ \mathcal{F}(J)|} \right),$$

với ký hiệu $\circ$ là nhân phần tử, I là `image`, J là `template`.

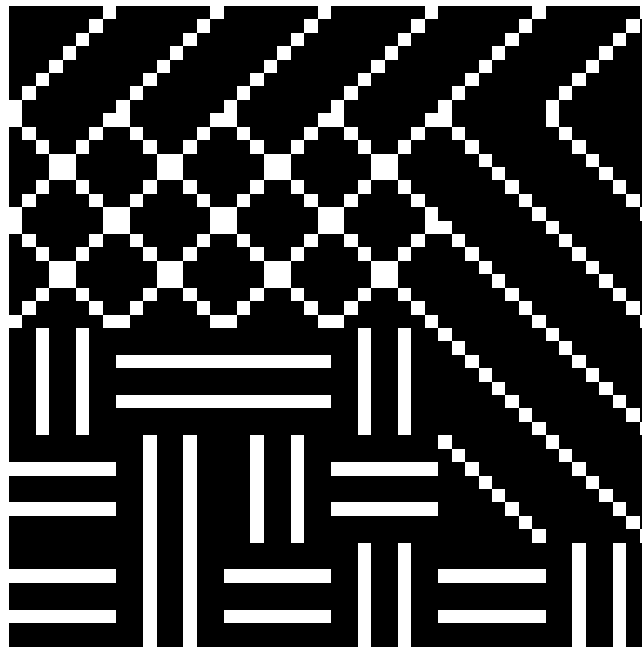
4. Lấy phần thực của C , tìm vị trí có giá trị lớn nhất.
5. Vẽ khung đánh dấu vị trí đó lên ảnh gốc.

Thực nghiệm: dùng `barton2-window.jpg` làm `template`, `barton2.jpg` làm `image`. So sánh *thời gian chạy* và *kết quả* với phương pháp template matching không dùng Fourier đã thực hành trước.

12.1 Bài tập 1: (Convolution)

Đọc ảnh **edge** kích thước 48x48, định dạng **grayscale**, thiết kế kernel, sử dụng các phép biến đổi và thực hiện các yêu cầu sau:

- Chuyển đổi ảnh về dạng float với giá trị trong $[0, 1]$



- Xác định các phần chứa đường chéo phụ trên ảnh.
- Xác định các phần chứa đường chéo chính trên ảnh.
- Xác định các phần chứa hai đường thẳng đứng trong ảnh.
- Xác định các phần chứa hai đường thẳng ngang trong ảnh.
- Xác định các phần chứa tổ hợp hai đường chéo trong ảnh.

12.2 Bài tập 2: (Độ đo trong Bài tập classification)

Trong Bài tập classification, các độ đo hiệu suất giúp đánh giá mức độ tốt của mô hình phân loại. Cho bảng phân loại kết quả như sau:

Viết chương trình python thực hiện tính toán các độ đo sau:

Confusion Matrix		
	Dự đoán: Positive	Dự đoán: Negative
Thực tế: Positive	TP = 50	FN = 10
Thực tế: Negative	FP = 5	TN = 35

- Tính độ chính xác (Accuracy): $Accuracy = \frac{TP+TN}{TP+FP+TN+FN}$
- Tính Precision (Độ chính xác theo lớp Positive): $Precision = \frac{TP}{TP+FP}$
- Tính Recall (Độ nhạy của lớp Positive hay là khả năng bỏ sót mẫu Positive): $Recall = \frac{TP}{TP+FN}$
- F1-score: $F1 - score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$

Nhận xét: nhận xét kết quả đạt được của mô hình trên, đánh giá độ tốt của mô hình trong ngữ cảnh cụ thể liên quan tới y tế.

12.3 Bài tập 3 *: (Two Layer Net)

Chỉ sử dụng **numpy**, xây dựng một mạng **Feed forward Neural Network** gồm 2 layer 128 node, hàm kích hoạt activation ReLU.

Thông tin các thành phần được định nghĩa như sau:

1. Hàm mục tiêu: Softmax $L = -\frac{1}{N} \sum_{i=1}^N \log \frac{e^{s_{y_i}}}{\sum_{i=1}^N e^{s_{y_i}}}$ với s_y là score của lớp thứ i.
2. Thuật toán tối ưu Batch Gradient Descent (BGD).
3. Learning rate fix cố định $\alpha = 0.01$.

Thực hiện yêu cầu sau:

- Thử nghiệm kết quả trên bộ dữ liệu giả lập random theo code sau: Trong đó:

```
import numpy as np
np.random.seed(0)
N, D, C = 200, 4, 3
X = np.random.randn(N, D)
y = np.random.randint(C, size=N)
```

– N: Số quan sát (giả thiết độc lập)

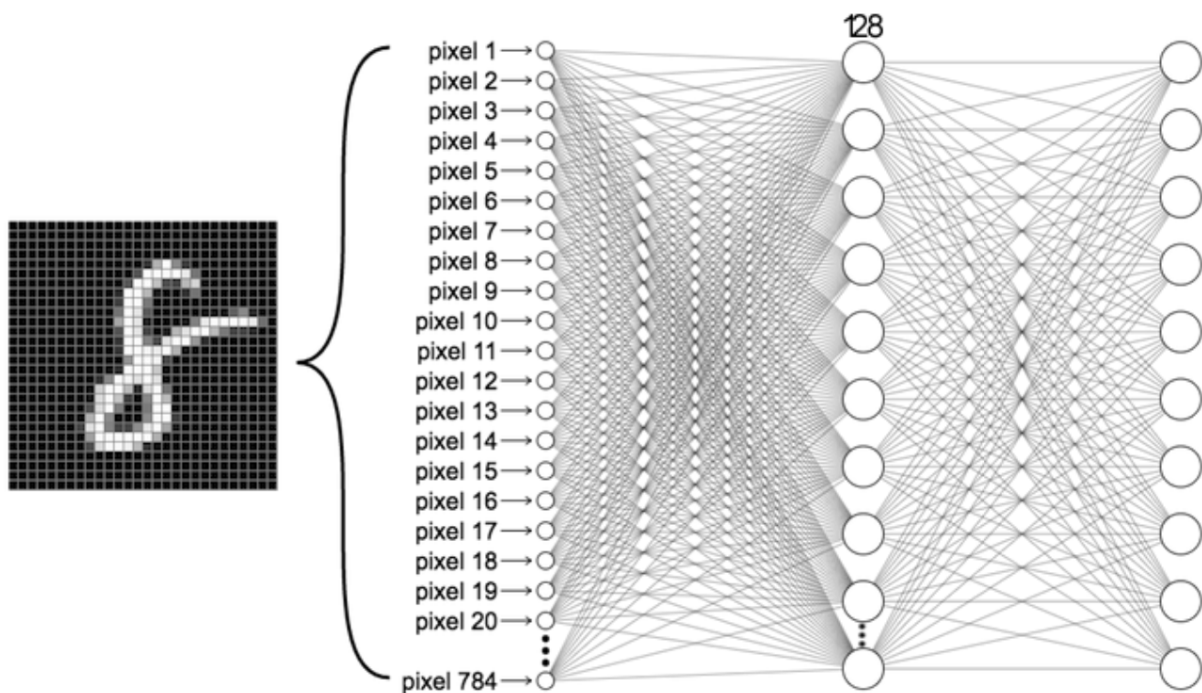
- D: Số chiều đặc trưng
- C: Số lớp classification
- Thử nghiệm mô hình trên tập data CIFAR-10.

12.4 Bài tập 4: MNIST (Modified National Institute of Standards and Technology)

MNIST (Modified National Institute of Standards and Technology) là một tập dữ liệu nổi tiếng và cơ bản trong lĩnh vực Machine Learning và Deep Learning, thường được sử dụng để thử nghiệm và phát triển các thuật toán nhận dạng mẫu (pattern recognition) hoặc phân loại.

Yêu cầu:

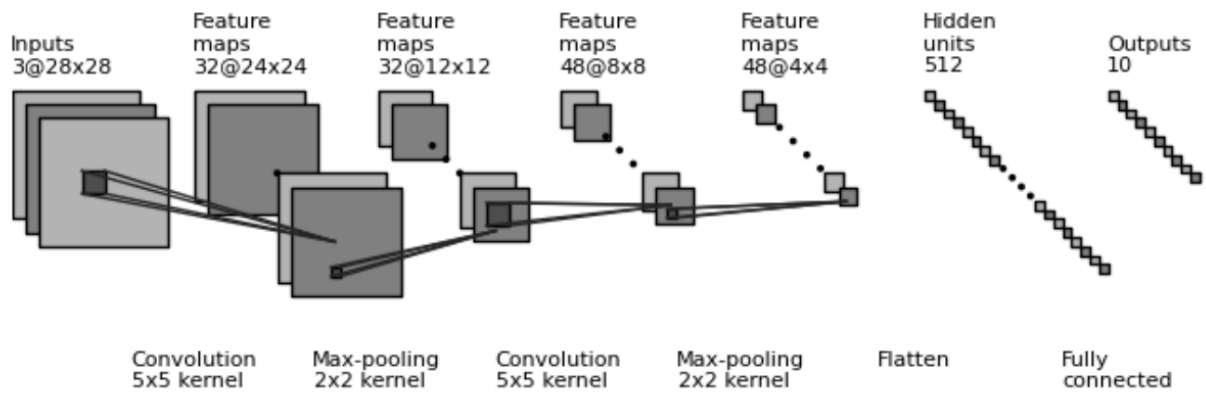
1. Tìm hiểu về bộ dữ liệu MNIST cho Bài tập classification.
2. Dựa vào những kiến thức đã học, xây dựng một mô hình mạng neural thực hiện tác vụ nhận diện chữ viết tay trên tập dữ liệu MNIST. (Sử dụng framework **TensorFlow** hoặc **PyTorch**). Quan sát trực quan các mô hình sau:
 - Mạng neural lan truyền thẳng (Feedforward Neural Network - FFN) sử dụng 2 lớp hidden layer trong Bài tập phân loại chữ viết.



- Cấu trúc mạng CNN cho Bài tập phân loại ảnh.

3. Chú ý:

- Colab Notebook (Kaggle) hỗ trợ sử dụng GPU, giúp tăng tốc độ huấn luyện mô hình lên đáng kể.



- Cả TensorFlow và PyTorch đều có sẵn tập dữ liệu MNIST.

4. Kiểm tra kết quả mô hình bằng các độ đo đã học (Accuracy, Precision, Recall, F1-score, ...) và nhận xét về kết quả thu được.

Xử lý ảnh

Deep learning Advance: CNN model

Giảng viên: Đỗ Thanh Hà

VNU - University of Science

13.1 Bài tập 1: (Tập dữ liệu Imagenette2)

Bộ dữ liệu Imagenette2 được phát hành bởi nhóm fastai là một phiên bản nhỏ của ImageNet, được tạo ra để thử nghiệm các mô hình học sâu về phân loại ảnh mà không cần tải toàn bộ ImageNet. đây là bộ dữ liệu hoàn toàn public *phục vụ cho học tập và giảng dạy* ([link](#)).

Trong bài tập này, sinh viên sử dụng phiên bản imagenette2 (ảnh kích thước 320x320 pixel).

Bộ dữ liệu gồm 10 lớp với nhãn là mã định danh dạng **n0xxxxxxx**.

Ví dụ: Khi giải nén bộ dữ liệu, sinh viên có thể thấy một thư mục có tên **n03394916**. Đây là mã WNID tương ứng với lớp “French Horn”.

Bảng ánh xạ giữa WNID và tên lớp:

WNID	Tên lớp
n01440764	Tench
n02102040	English Springer
n02979186	Cassette Player
n03000684	Chain Saw
n03028079	Church
n03394916	French Horn
n03417042	Garbage Truck
n03425413	Gas Pump
n03445777	Golf Ball
n03888257	Parachute

Dành ra vài phút để đọc hiểu dữ liệu và thực hiện các yêu cầu sau:

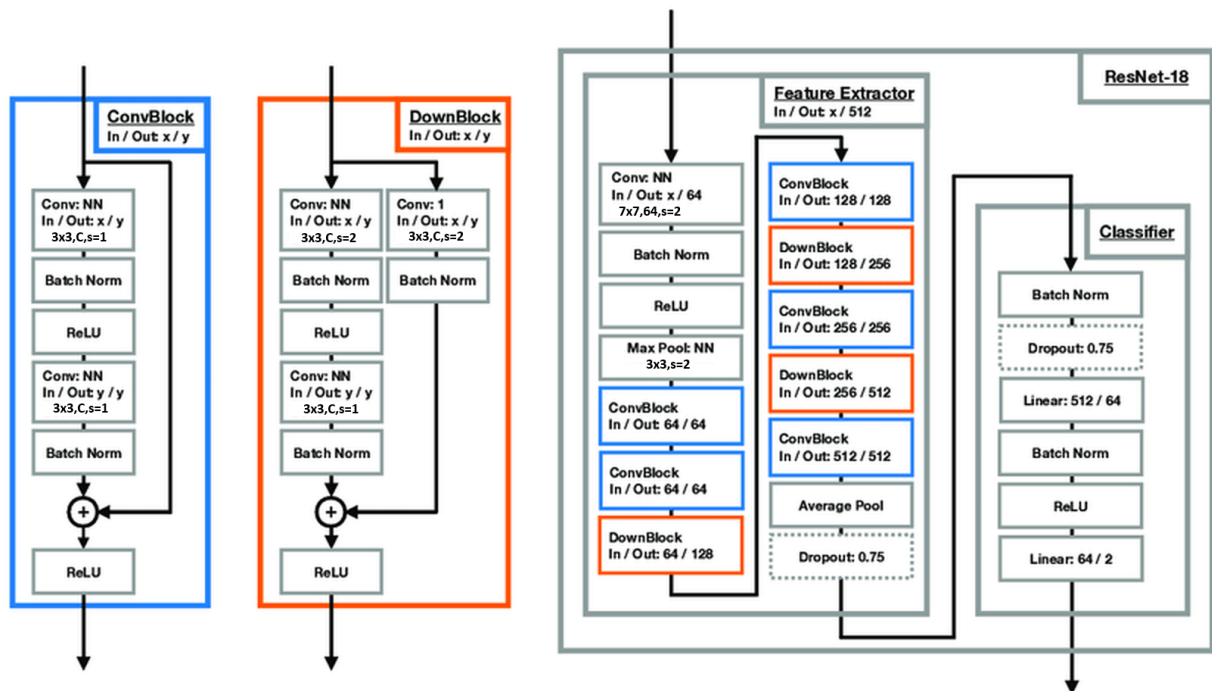
- Với mỗi lớp hiện có, hiển thị 3 ảnh và nhãn lên màn hình để trực quan hóa dữ liệu.
- Vẽ biểu đồ cột thống kê số lượng mẫu của mỗi lớp, đưa ra nhận xét.
- Thực hiện tăng cường dữ liệu để làm đa dạng hóa bộ dữ liệu, hiển thị kết quả thu được. Sinh viên áp dụng ít nhất 3 kỹ thuật tăng cường dữ liệu dưới đây:
 - Lật ngang ảnh (Horizontal Flip).
 - Xoay ảnh ngẫu nhiên (Random Rotation từ -15 độ đến 15 độ).
 - Thay đổi độ sáng hoặc độ tương phản ảnh (Brightness/Contrast).

Chú ý: Sử dụng PyTorch hoặc TensorFlow để xây dựng Dataset/Dataloader, trình bày mã nguồn rõ ràng, có bình luận từng bước thực hiện.

13.2 Bài tập 2: (Kiến trúc ResNet)

ResNet (Residual Network) là một kiến trúc mạng nơ-ron sâu do Microsoft giới thiệu năm 2015, nổi bật với cơ chế "kết nối tắt" (skip connection). Cơ chế này giúp truyền trực tiếp đầu vào của một lớp đến đầu ra của lớp sau, giúp mô hình giảm hiện tượng mất thông tin và gradient vanishing khi mạng trở nên rất sâu.

Cho hình vẽ dưới đây và sử dụng framework bất kỳ, thực hiện yêu cầu sau:



- Dựa vào hình vẽ, xây dựng chính xác kiến trúc mạng ResNet-18. Sinh viên cần chú ý rõ ràng các lớp ConvBlock, DownBlock, số lượng các lớp Convolution, kích thước kernel, stride và các skip connection giữa các tầng.
- Hiển thị kiến trúc của mô hình vừa xây dựng bằng các công cụ visualize phù hợp như:
 - Với **PyTorch**, sinh viên có thể dùng thư viện như `torchsummary`, `torchviz` để hiển thị kiến trúc và flow của mạng.
 - Với **TensorFlow**, sinh viên có thể dùng phương pháp `model.summary()` hoặc thư viện `TensorBoard` để hiển thị kiến trúc mạng.
- Chạy thử mô hình với dữ liệu đầu vào giả (dummy data) có kích thước đầu vào (32, 3, 224, 224) để xác nhận mô hình hoạt động đúng, và hiển thị kích thước đầu ra cuối cùng.

13.3 Bài tập 3: (Huấn luyện mô hình)

Sử dụng kiến trúc mạng ResNet-18 vừa xây dựng để thực hiện huấn luyện trên bộ dữ liệu Imagenette2-320.

- Chia bộ dữ liệu thành các tập huấn luyện, kiểm thử (đã có sẵn).
- Tải dữ liệu vào Dataset/Dataloader.
- Lựa chọn loss function là crossentropy, hàm tối ưu Adam (learning rate = $1e-3$) và batch size = 32.
- Huấn luyện mô hình trong tối thiểu 10 epochs. Theo dõi và lưu lại kết quả độ chính xác (accuracy) và mất mát (loss) trên tập huấn luyện và kiểm thử qua từng epoch.
- Vẽ đồ thị biểu diễn sự thay đổi của accuracy và loss qua các epochs. Đưa ra nhận xét về hiệu quả của việc huấn luyện (hiện tượng hội tụ, overfitting, underfitting,... nếu có) và cho biết các siêu tham số đã mang lại kết quả tối ưu nhất chưa.

Yêu cầu: Trình bày mã nguồn rõ ràng, chi tiết và có chú thích từng bước. Sinh viên có thể sử dụng GPU để hỗ trợ và tăng tốc huấn luyện.